

The Mandelbrot set

The Mandelbrot set is defined as the set of complex numbers c for which the iterative sequence: $z_{n+1} = z_n^2 + c$

with $z_0 = 0$, remains bounded (doesn't diverge to infinity) which we define as, $|z_{n+1}| \leq 2$ for all n .

So the first iterations of the sequence looks like this. $z_0 = 0$

$$z_1 = 0 + c = c$$

$$z_2 = c^2 + c$$

$$z_3 = (c^2 + c)^2 + c$$

...

The naive approach in Python

When calculating all the values in the Mandelbrot set, one could take a naive approach.

Disregarding any optimization choices and use default python lists and nested for loops:

```
In [1]: # Define the limits and parameters for the mandelbrot set.
# The typical view (limits) as defined by the problem description from moodl
x_min = -2.0
x_max = 1.0
y_min = -1.5
y_max = 1.5

# Minimum resolution to start with. Will be updated later to test scalabilit
width = 1024
height = 1024

# Fixed number of iterations taken from moodle.
max_iterations = 100

def mandelbrot(c, max_iterations):
    z = 0
    n = 0

    # Keep iterating as long as the magnitude of z is less than or equal to
    while abs(z) <= 2.0 and n < max_iterations:
        z = z*z + c
        n += 1
    return n

def generate_mandelbrot_set_naive():
    # Create a naive 2D list to hold the values.
```

```
mandelbrot_set = [[0 for _ in range(width)] for _ in range(height)]

# Loop through all pixels and determine how many iterations it takes for
for y in range(height):
    for x in range(width):
        real = x_min + (x / width) * (x_max - x_min)
        imag = y_min + (y / height) * (y_max - y_min)
        c = complex(real, imag)
        mandelbrot_set[y][x] = mandelbrot(c, max_iterations)

# Return iteration counts for each complex number.
return mandelbrot_set

mandelbrot_set_naive = generate_mandelbrot_set_naive()
naive_result = %timeit -o generate_mandelbrot_set_naive()
```

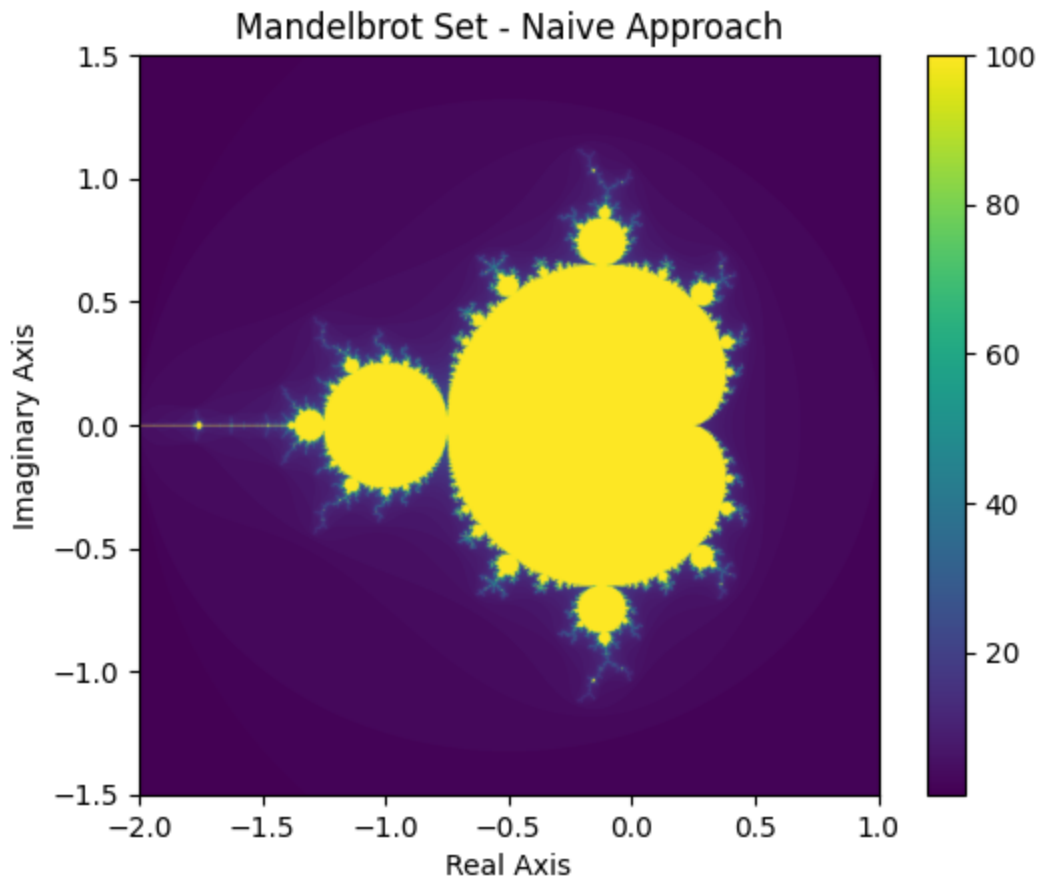
2.52 s ± 41.1 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

As we can see the naive approach results in each function call taking multiple seconds to complete at this scale (1024 x 1024).

Using a colorbar plot we can visualize the set and confirm that we get the expected result:

```
In [3]: import matplotlib.pyplot as plt

# Plotting the set using colorbar.
plt.imshow(mandelbrot_set_naive, extent=(x_min, x_max, y_min, y_max))
plt.colorbar()
plt.title("Mandelbrot Set - Naive Approach")
plt.xlabel("Real Axis")
plt.ylabel("Imaginary Axis")
plt.show()
```



Vectorized approach in Python using NumPy

Now that we have tried the naive approach we can start thinking about ways to optimize the approach. - and as listed by the problem description the first option that we try to implement is a vectorized approach.

In the vectorized approach we try to limit the amount of loops by replacing the (looped) iterations with calculations that can be done simultaneously with vector operations:

```
In [21]: import numpy as np

def generate_mandelbrot_set_vectorized():

    # Create a 2D array of complex numbers representing the points in the co
    real = np.linspace(x_min, x_max, width)
    imag = np.linspace(y_min, y_max, height)

    # Define all the complex numbers in the grid.
    c = np.array([[complex(r, i) for r in real] for i in imag])

    # Initialize an array to hold the number of iterations for each point.
    mandelbrot_set = np.zeros(c.shape, dtype=int)

    # Create an array to hold the current values of z for each point.
    z = np.zeros(c.shape, dtype=complex)
```

```
# Iterate up to max_iterations times, updating all points simultaneously
for n in range(max_iterations):
    mask = np.abs(z) <= 2.0 # Only update points that haven't diverged
    z[mask] = z[mask] * z[mask] + c[mask] # Update z for those points.
    mandelbrot_set[mask] = n # Update the iteration count for those poi

# Return iteration counts for each complex number.
return mandelbrot_set

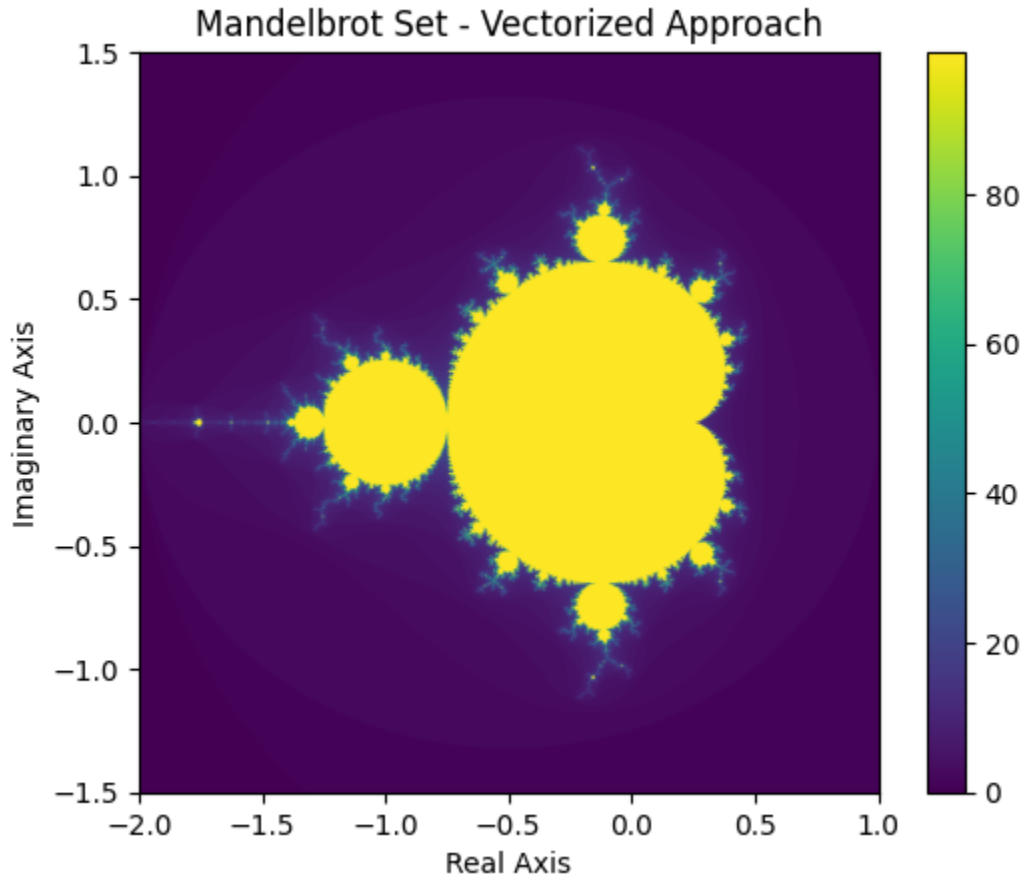
mandelbrot_set_vectorized = generate_mandelbrot_set_vectorized()
vectorized_result = %timeit -o generate_mandelbrot_set_vectorized()
```

917 ms $\pm$ 81.4 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

And now we see that a function call takes just shy of 1 second to complete at the same scale (1024 x 1024).

Again, we can use a colorbar plot we can visualize the set and confirm that we get the expected result:

```
In [ ]: # Plotting the new set using colorbar.
plt.imshow(mandelbrot_set_vectorized, extent=(x_min, x_max, y_min, y_max))
plt.colorbar()
plt.title("Mandelbrot Set - Vectorized Approach")
plt.xlabel("Real Axis")
plt.ylabel("Imaginary Axis")
plt.show()
```



Precompiled approach using Numba and JIT (Just In Time compilation)

Now that we have tried to optimize the approach using vector calculations, we can also try to just let the compiler optimize for us using JIT. -Specifically numba.jit():

```
In [6]: import numba

@numba.jit(nopython=True)
def generate_mandelbrot_set_jit():

    # Define helper function here to avoid numba issues with nested function
    def mandelbrot(c, max_iterations):
        z = 0
        n = 0
```

```

    # Keep iterating as long as the magnitude of z is less than or equal to 2.0
    while abs(z) <= 2.0 and n < max_iterations:
        z = z*z + c
        n += 1
    return n

# Create a naive 2D list to hold the values.
mandelbrot_set = [[0 for _ in range(width)] for _ in range(height)]

# Loop through all pixels and determine how many iterations it takes for
for y in range(height):
    for x in range(width):
        real = x_min + (x / width) * (x_max - x_min)
        imag = y_min + (y / height) * (y_max - y_min)
        c = complex(real, imag)
        mandelbrot_set[y][x] = mandelbrot(c, max_iterations)

# Return iteration counts for each complex number.
return mandelbrot_set

mandelbrot_set_jit = generate_mandelbrot_set_jit()
jit_result = %timeit -o generate_mandelbrot_set_jit()

```

211 ms ± 3.09 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

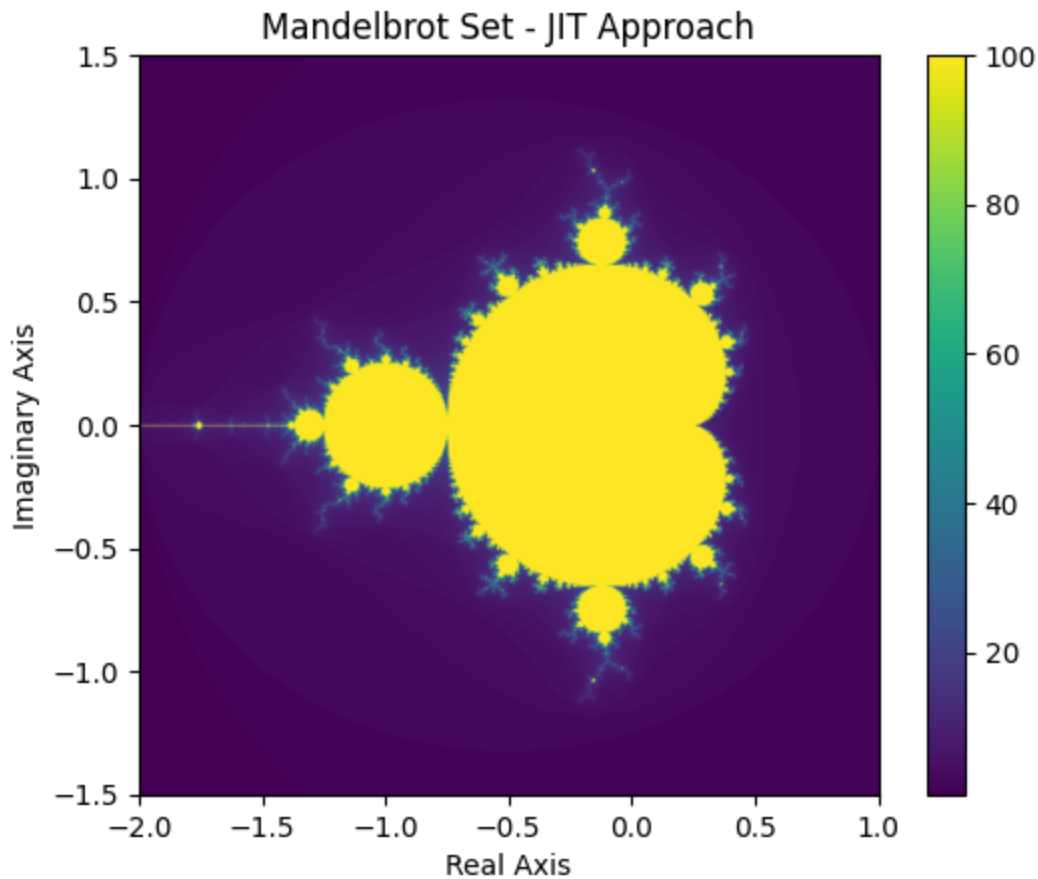
And here we see even more improvement compared to the naive approach. -eventhough we just compiled it.

Again, we can use a colorbar plot we can visualize the set and confirm that we get the expected result:

```

In [7]: # Plotting the JIT set using colorbar.
plt.imshow(mandelbrot_set_jit, extent=(x_min, x_max, y_min, y_max))
plt.colorbar()
plt.title("Mandelbrot Set - JIT Approach")
plt.xlabel("Real Axis")
plt.ylabel("Imaginary Axis")
plt.show()

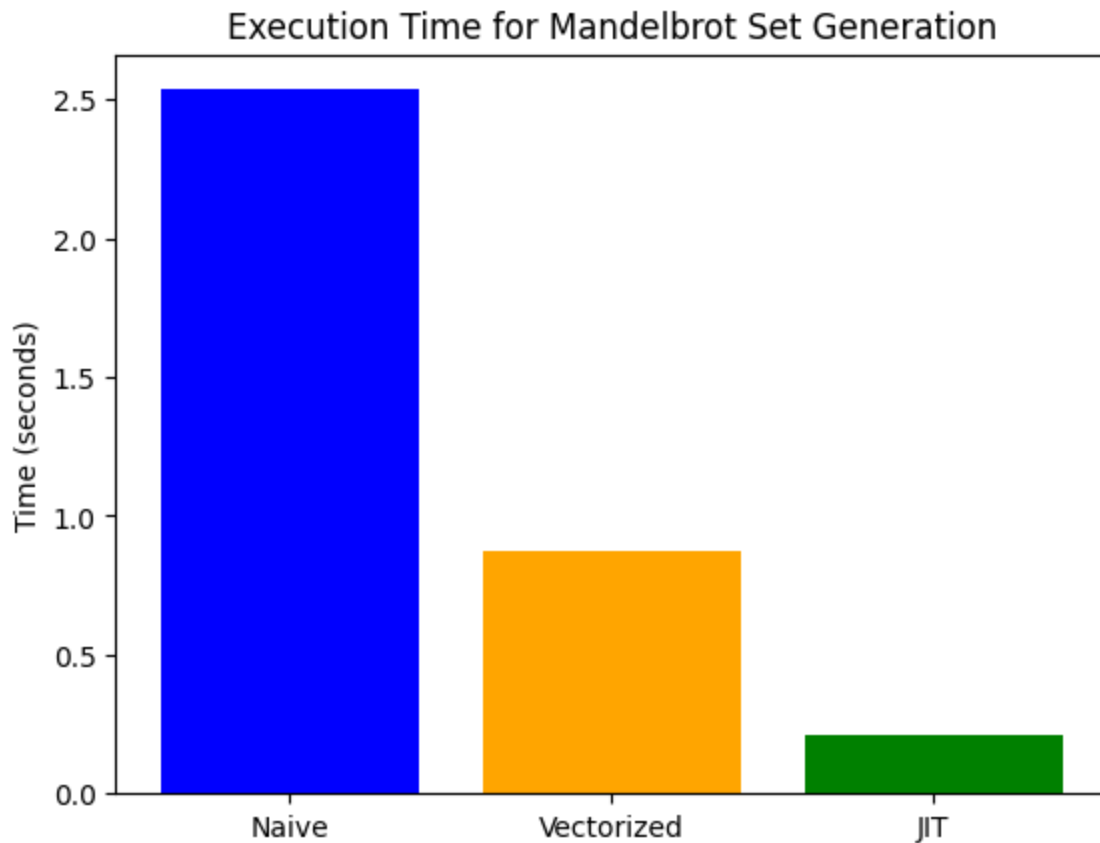
```



Performance analysis

Now that we have implemented all three approaches and measured the time it takes for each, we can compare them:

```
In [8]: # Plot the time results for the three approaches.
approaches = ['Naive', 'Vectorized', 'JIT']
times = [naive_result.average, vectorized_result.average, jit_result.average]
plt.bar(approaches, times, color=['blue', 'orange', 'green'])
plt.ylabel('Time (seconds)')
plt.title('Execution Time for Mandelbrot Set Generation')
plt.show()
```



The problem description from moodle also states that we should calculate the speedups which have been presented as: $S_p = T_1/T_p$. - but since none of the implementation has been done using parallel computing i will adjust the meaning of the variables

T_1 : naive time, T_p : vectorized or JIT time

```
In [9]: speedup_vectorized = naive_result.average / vectorized_result.average
        speedup_jit = naive_result.average / jit_result.average

        print(f"Speedup of Vectorized approach over Naive: {speedup_vectorized:.2f}x")
        print(f"Speedup of JIT approach over Naive: {speedup_jit:.2f}x")
```

```
Speedup of Vectorized approach over Naive: 2.91x
Speedup of JIT approach over Naive: 12.03x
```


The difference in performance from naive to vectorized comes from:

1. Optimized instructions for vectors vs for loops: The instruction set that a processors have can be better optimized for vector calculations than for loops.
2. Cache misses: Since are using the native python list in the naive approach, our memory layout of the lists might be non-contiguous (strided) or contiguous (unit stride).

The difference in performance from vectorized to JIT comes from:

1. Compiler optimizations: Beyond the optimization from using vectors, there might be unknown optimizations that can be access when using a compiler.

Scaling analysis

```
In [10]: # Create list of scales
scales = [1024, 1536, 2048, 3072, 4096]

# Run the three approaches for each scale and store the results.
naive_results = []
vectorized_results = []
jit_results = []
for scale in scales:
    width = height = scale # Update the resolution for the current scale.

    # Run and store results for each approach.
    naive_result = %timeit -o generate_mandelbrot_set_naive()
    naive_results.append(naive_result)
```

```
2.48 s ± 11.7 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
5.45 s ± 32.1 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
9.64 s ± 13.8 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
21.8 s ± 91.2 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
38.6 s ± 192 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
In [11]: for scale in scales:
    width = height = scale # Update the resolution for the current scale.

    # Run and store results for each approach.
    vectorized_result = %timeit -o generate_mandelbrot_set_vectorized()
    vectorized_results.append(vectorized_result)
```

```
879 ms ± 7.51 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
2.33 s ± 99.5 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
4.49 s ± 15.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
12.5 s ± 33.6 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
23.8 s ± 18.1 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
In [12]: for scale in scales:
    width = height = scale # Update the resolution for the current scale.

    # Run and store results for each approach.
```

```
jit_result = %timeit -o generate_mandelbrot_set_jit()
jit_results.append(jit_result)
```

213 ms ± 663 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
 217 ms ± 7.35 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
 213 ms ± 513 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
 214 ms ± 1.02 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
 213 ms ± 393 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)

```
In [ ]: print("Scalability results:")
        print("Naive approach:")
        for i, scale in enumerate(scales):
            print(f"Scale {scale}x{scale}: {naive_results[i].average:.4f} seconds")
        print("\nVectorized approach:")
        for i, scale in enumerate(scales):
            print(f"Scale {scale}x{scale}: {vectorized_results[i].average:.4f} seconds")
        print("\nJIT approach:")
        for i, scale in enumerate(scales):
            print(f"Scale {scale}x{scale}: {jit_results[i].average:.4f} seconds")
```

Scalability results:

Naive approach:

Scale 1024x1024: 2.4794 seconds
 Scale 1536x1536: 5.4540 seconds
 Scale 2048x2048: 9.6427 seconds
 Scale 3072x3072: 21.7664 seconds
 Scale 4096x4096: 38.5979 seconds

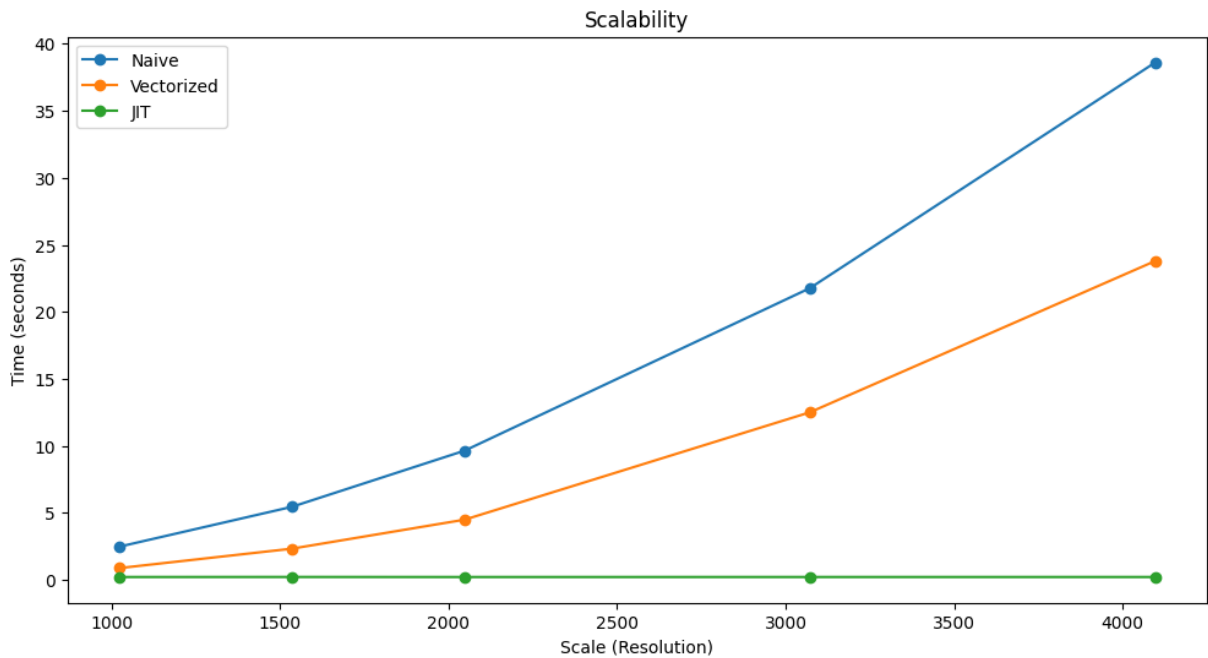
Vectorized approach:

Scale 1024x1024: 0.8792 seconds
 Scale 1536x1536: 2.3291 seconds
 Scale 2048x2048: 4.4932 seconds
 Scale 3072x3072: 12.5029 seconds
 Scale 4096x4096: 23.7880 seconds

JIT approach:

Scale 1024x1024: 0.2132 seconds
 Scale 1536x1536: 0.2168 seconds
 Scale 2048x2048: 0.2128 seconds
 Scale 3072x3072: 0.2136 seconds
 Scale 4096x4096: 0.2132 seconds

```
In [15]: # Plotting the scalability results for each approach.
        plt.figure(figsize=(12, 6))
        plt.plot(scales, [result.average for result in naive_results], label='Naive')
        plt.plot(scales, [result.average for result in vectorized_results], label='V')
        plt.plot(scales, [result.average for result in jit_results], label='JIT', ma
        plt.xlabel('Scale (Resolution)')
        plt.ylabel('Time (seconds)')
        plt.title('Scalability')
        plt.legend()
        plt.show()
```



```
In [ ]: # Export results to CSV as requested in the problem description.
import csv

with open('mandelbrot_scalability_results_naive.csv', mode='w', newline='')
    writer = csv.writer(file)
    writer.writerow(['Scale', 'Time (seconds)'])
    for i, scale in enumerate(scales):
        writer.writerow([f"{scale}x{scale}", f"{naive_results[i].average:.4f}"])

with open('mandelbrot_scalability_results_vectorized.csv', mode='w', newline='')
    writer = csv.writer(file)
    writer.writerow(['Scale', 'Time (seconds)'])
    for i, scale in enumerate(scales):
        writer.writerow([f"{scale}x{scale}", f"{vectorized_results[i].average:.4f}"])

with open('mandelbrot_scalability_results_jit.csv', mode='w', newline='') as
    writer = csv.writer(file)
    writer.writerow(['Scale', 'Time (seconds)'])
    for i, scale in enumerate(scales):
        writer.writerow([f"{scale}x{scale}", f"{jit_results[i].average:.4f}"])
```